

WebSocket

23 July 2026 01:33 PM

What are WebSockets?

WebSockets are a **communication protocol** that enables **real-time, two-way (bidirectional) communication** between a client and a server over a **single persistent TCP connection**.

Unlike HTTP, where the client sends a request and waits for a response, WebSockets allow **both the client and the server to send messages at any time** after the connection is established.

Example:

- 🗨️ WhatsApp messages
- 🎮 Multiplayer games
- 📊 Live stock prices
- 🔔 Live notifications

Why Do We Need WebSockets?

Traditional HTTP is **request-response based**.

Client ---- Request ----> Server

Client <--- Response ---- Server

(Connection closes)

If the server wants to notify the client later, it **cannot**, because the connection has already closed.

With WebSockets:

Client ⇄ WebSocket Connection ⇄ Server

The connection **stays open**, allowing both sides to exchange data instantly.

How WebSockets Work

Step 1: Handshake

The client sends a normal HTTP request with an **Upgrade: websocket** header.

Client

|

HTTP GET

Upgrade: websocket

|

If the server supports WebSockets, it responds with:

HTTP/1.1 101 Switching Protocols

The protocol is upgraded from HTTP to WebSocket.

Step 2: Connection Established

After the handshake:

Client

↑↓

WebSocket

↑↓

Server

The connection remains open until either side closes it.

Step 3: Data Transfer

Both client and server can send messages independently.

Client

↑↓

Message

Server

Messages are sent as **frames**, which have very small overhead (as little as 2 bytes).

Step 4: Connection Closure

Either the client or server can close the connection by sending a **Close Frame**.

Why Use WebSockets?

1. Real-Time Communication

Messages are delivered instantly.

Examples:

- Chat apps
- Online games
- Live dashboards

2. Low Latency

No need to create a new connection for every request.
Communication is almost instantaneous.

3. Efficient

After the initial handshake, only small frames are exchanged.
Less bandwidth compared to HTTP polling.

4. Bidirectional Communication

Both client and server can initiate communication.
Example:

Server → Client
"New message received"

Client → Server
"Reply message"

5. Persistent Connection

One connection remains open until closed.
No repeated connection setup.

WebSockets vs Other Communication Methods

1. HTTP

How it works

Client → Request
Server → Response
Connection Closed

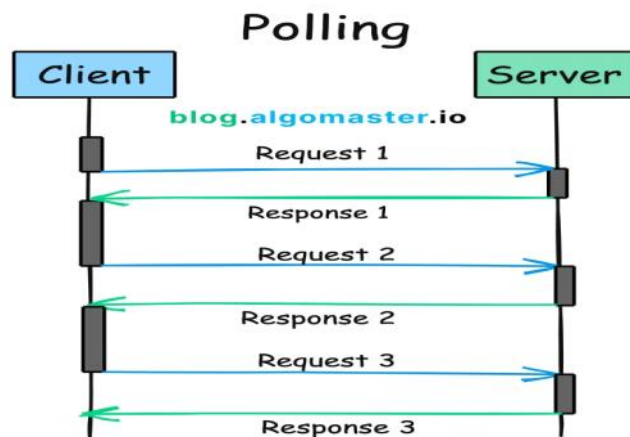
Advantages

- Simple
- Stateless

Disadvantages

- Cannot push data to clients.
- High latency for real-time apps.

2. Polling



Client repeatedly asks the server:

Client → Any updates?

Server → No

Client → Any updates?

Server → No

Client → Any updates?

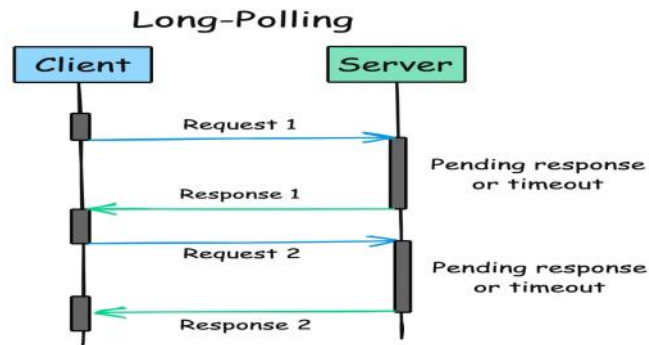
Server → Yes

Problems

- Many unnecessary requests.

- Wastes bandwidth.
- Delayed updates.

3. Long Polling



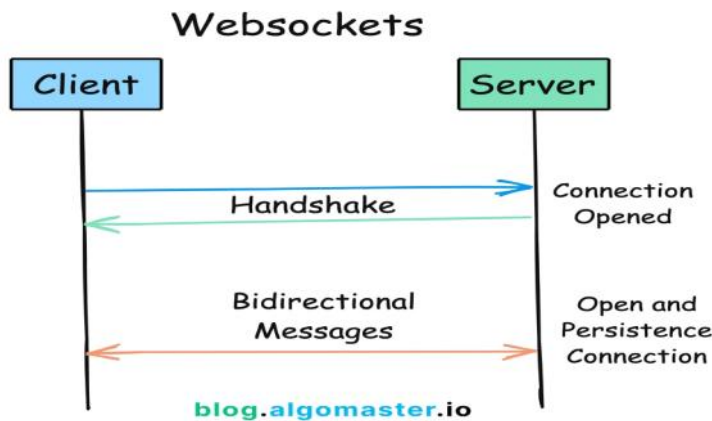
Client sends one request.
Server waits until data is available.

Client → Request

(Server waits...)
Server → New Data

Connection Closed
Client sends another request
Better than polling but still creates a new HTTP request after every response.

4. WebSockets



Client
↕
Persistent Connection
↕
Server
Both sides communicate whenever needed.
No repeated HTTP requests.

Challenges of WebSockets

1. Proxy & Firewall Issues

Some proxies or firewalls may block WebSocket connections.

2. Scalability

Thousands or millions of open connections require:

- Load Balancers
- Distributed WebSocket Servers

3. Fallback Mechanism

Some clients may not support WebSockets.

Fallback options:

- Long Polling

- HTTP Polling

4. Network Reliability

Connections can drop due to network issues.

Solutions:

- Automatic reconnection
- Heartbeat (Ping/Pong)

5. Security

Use secure WebSockets:

wss://
instead of

ws://

Also:

- Authenticate users.
- Validate incoming data.
- Protect against DDoS attacks.

IMPLEMENTATION:

Server side(node.js)

```
const WebSocket = require('ws');
const server = new WebSocket.Server({ port: 8080 });

server.on('connection', socket => {
  console.log('New connection established');

  // Send a message to the client
  socket.send('Hello from the server!');

  // Receive messages from the client
  socket.on('message', message => {
    console.log(`Received: ${message}`);
    // Echo the message back to the client
    socket.send(`You said: ${message}`);
  });

  // Handle connection closure
  socket.on('close', () => {
    console.log('Connection closed');
  });
});
```

Client Side(js)

```
const socket = new WebSocket('ws://localhost:8080');

// Connection opened
socket.addEventListener('open', event => {
  console.log('Connected to the server');
  socket.send('Hello, Server!');
});

// Listen for messages
socket.addEventListener('message', event => {
  console.log('Message from server: ', event.data);
});

// Handle connection closure
socket.addEventListener('close', event => {
  console.log('Connection closed');
});
```

Where Are WebSockets Used?

- 💬 Chat Applications
- 👥 Real-Time Collaboration
- 🔔 Live Notifications
- 🎮 Multiplayer Game
- 📊 Financial Applications
- 🌐 IoT Applications
- 📺 Live Streaming

Advantages

- ☒ Real-time communication
- ☒ Low latency
- ☒ Persistent connection
- ☒ Bidirectional communication
- ☒ Low bandwidth usage after handshake

Disadvantages

- ✗ Harder to scale
- ✗ Requires connection management
- ✗ Proxy/firewall compatibility issues
- ✗ Needs reconnection logic
- ✗ More complex than HTTP